Accelerating LWE-Based Post-Quantum Cryptography with Approximate Computing

Diamante Simone Crescenzo[1], Emanuele Valea[1], Alberto Bosio[2]
[1]Univ. Grenoble Alpes, CEA, List, F-38000 Grenoble, France
[2]Institut des Nanotechnologies de Lyon, École Centrale de Lyon, Lyon, France

***Abstract*—Conventional cryptographic algorithms rely on hard mathematical problems to ensure an appropriate level of security. However, with the advent of quantum computing, classical cryptographic algorithms are expected to become vulnerable. For this reason, Post-Quantum Cryptography (PQC) algorithms have emerged as a response, being designed to resist quantum attacks. Most PQC algorithms rely on the Learning With Errors (LWE) problem, where generating pseudo-random controlled errors is crucial. A well-known solution is the use of hash functions followed by error samplers, implemented according to specific error distributions, whose implementation is challenging. This paper provides a proof of concept demonstrating how Approximate Computing (AxC) can be exploited in LWE-based cryptographic algorithms to alleviate this implementation bottleneck. The main idea is to use AxC circuits to run some of the algorithm's operations, introducing the required error for free thanks to the approximation. Our key contribution is demonstrating how AxC techniques can be effectively applied to LWE-based algorithms, highlighting a novel approach to generating and introducing the error. This concept has proven effective in an approximate implementation of the FrodoKEM algorithm, where we achieve a 50.3% reduction in the need for Gaussian sampling. Additionally, we observe a performance improvement of 2.19%, which further supports the feasibility of this approach. Overall, this work introduces and validates a new design direction for LWE-based cryptography through AxC, opening the way for further research.**

***Index Terms*—Post-Quantum Cryptography (PQC), Learning With Errors (LWE), Approximate Computing (AxC), Error Sampling, AxPQC.**

I. Introduction

It is well established that cryptography is essential for securing communication in the digital era. Encryption and authentication algorithms ensure data integrity and confidentiality, relying on hard mathematical problems for security. However, quantum computing threatens conventional schemes [1], necessitating *Post-Quantum Cryptography* (PQC) designed to resist quantum attacks. FrodoKEM [2] is one such algorithm, based on the *Learning With Errors* (LWE) problem [3].

The LWE problem involves recovering a secret vector from linear equations perturbed by small, structured errors, typically drawn from a discrete Gaussian or other bounded distributions. Its security derives from worst-case lattice problems, such as the *Shortest Vector Problem* (SVP) [3], which remain hard to solve even for quantum algorithms. The difficulty of distinguishing perturbed linear relations from random noise underpins LWE-based cryptography.

Noise generation poses challenges due to its complex sampling chains, typically consisting of three steps: generating a random seed via a True Random Number Generator (TRNG); expanding it into a pseudo-random sequence using a cryptographically secure hash function (often following the NIST FIPS 202 standard [4], based on Keccak [5]); and transforming the uniform distribution into the required Gaussian distribution using a dedicated *sampler* [6]. This process is computationally intensive, with dedicated hardware solutions still emerging [7].

Approximate Computing (AxC) offers a promising alternative by trading computational accuracy for efficiency, such as reduced area, power consumption, or faster execution, making it well-suited for LWE-based schemes where controlled error generation is necessary [8].

This paper explores AxC as a means to optimize LWE-based algorithms by eliminating the explicit error-generation chain. Instead of adding separate error terms, we approximate the matrix-vector product computation itself. FrodoKEM is chosen as a case study, as it directly stems from the standard LWE problem, facilitating a general discussion on error introduction. We provide a proof-of-concept FrodoKEM implementation that incorporates errors via AxC-based digital operators, leveraging approximate adders from EvoApproxLib [9] after characterizing their error distribution. The approximate adders are emulated in software to evaluate functional correctness, while performance results assume a hardware implementation with equivalent performance to precise adders.

Our key contribution is demonstrating how AxC can effectively introduce errors in LWE-based cryptography by integrating them directly into the computation, which allows us to reduce the required Gaussian sampling by 50.3%. Applied to FrodoKEM, this proof-of-concept approach yields a 2.19% reduction in key exchange execution time. These results support the feasibility of using AxC as a novel tool for simplifying error generation in LWE-based schemes.

The remainder of this paper is organized as follows: Section II provides background on Approximate Computing (AxC) and the Learning With Errors (LWE) problem. Section III details our methodology for integrating AxC into LWE-based schemes. Section IV presents experimental results, while Section V discusses broader implications and extensions. Finally, Section VI concludes the paper and outlines future work.

II. LWE, Cryptographic Schemes, and AxC

A. Learning With Errors

To describe the Learning With Errors (LWE) problem, introduced for the first time in [3], we will use the convenient

tools of linear algebra. Let us define the *secret vector* s of length n, and a *coefficient matrix* $\mathbf{A}$ of size $m \times n$, with $m > n$, and each element a_{ij} is sampled from a *uniform distribution*. The inner product $\mathbf{A} \cdot s$ generates the *result vector* b. This setup gives rise to an over-determined system of linear equations as follows:

$$\mathbf{A} \cdot s = b \tag{1}$$

It can be easily seen that it will only take n independent equations from Equation 1 to solve for s, thus retrieving the secret by Gaussian elimination. However, it is possible to slightly modify the left side of Equation 1 to significantly increase the level of difficulty in deducing the secret information by introducing *errors*. This is done by adding a vector e of dimension m, whose elements are small values (i.e., close to zero), sampled from a Gaussian distribution. The distribution is centered at zero, and to keep the errors small and controlled, a specific *standard deviation* σ must be maintained. Our system then becomes:

$$\mathbf{A} \cdot s + e = b \tag{2}$$

The obtained noisy system of linear equations is a hard problem, making s a *secret* that is extremely difficult for an attacker to deduce given $\mathbf{A}$ and b.

The LWE problem can serve as a basis for a *Public Key Cryptosystem*. Considering s as the *private key* and the concatenation of $\mathbf{A}$ and b as the *public key*, three functions are defined:

- **Key Generation**: Algorithm 1 provides a high-level overview of the necessary steps to generate both the public and private key. Matrix $\mathbf{A}$ is sampled uniformly at random, while the vectors s and e can be sampled from any distribution that respects the given LWE-based problem specifications. The SAMPLEVECTOR() procedure is a generation chain starting with pseudo-random data extracted via secure hash functions. This uniform data is then re-shaped by means of *samplers*. The public vector b is then computed as the matrix-vector product $b = \mathbf{A}s + e$.
- **Encryption**: as shown in Algorithm 2 encrypting a message takes the previously generated public key, together with a new ephemeral secret s', and two error vectors e' and e''. The concatenation of the resulting u and v vectors is the $ciphertext$.
- **Decryption**: Algorithm 3 outlines the steps to retrieve the original message from the ciphertext using the secret key. During encryption, small errors were added, and these errors are still present in the decrypted message. The DECODE() procedure plays a key role in correcting them by rounding the noisy value to the nearest valid message. As long as the errors are small enough, the original message can be recovered correctly.

In the literature, some algorithms implement the LWE problem in its standard form, such as FrodoKEM [2], which is discussed in subsection II-B and employed in our work.

Algorithm 1 LWE - Key Generation

Procedure KEYGENERATION()
  $\mathbf{A} \leftarrow$ GENERATEPUBLICMATRIX()
  $s \leftarrow$ SAMPLEVECTOR()
  $e \leftarrow$ SAMPLEVECTOR()
  $b \leftarrow \mathbf{A} \cdot s + e$
  public_key $\leftarrow (\mathbf{A}, b)$
  private_key $\leftarrow s$
  return (public_key, private_key)
End Procedure

Algorithm 2 LWE - Encryption

Procedure ENCRYPT(public_key, message m)
  $(\mathbf{A}, b) \leftarrow$ public_key
  $s' \leftarrow$ SAMPLEVECTOR()
  $e' \leftarrow$ SAMPLEVECTOR()
  $e'' \leftarrow$ SAMPLEVECTOR()
  $u \leftarrow s' \cdot \mathbf{A} + e'$
  $v \leftarrow s' \cdot v + e'' +$ ENCODE(m)
  ciphertext $\leftarrow (u, v)$
  return ciphertext
End Procedure

B. FrodoKEM

FrodoKEM [2] is a practical implementation of the general LWE problem, translating its theoretical foundations into a practical cryptographic scheme. We focus on FrodoKEM-640, where matrix $\mathbf{A}$ consists of 640×640 pseudo-random 15-bit values generated via FIPS 202 [4] hash functions.

The secret and error matrices, $\mathbf{s}$ and $\mathbf{e}$ (640×8), are produced similarly and passed to a Gaussian sampler to ensure a discrete, zero-centered distribution with standard deviation $\sigma = 2.8$. The same applies to $\mathbf{s}'$ and $\mathbf{e}'$ (8×640) and $\mathbf{e}''$ (8×8), which are used in Algorithm 2 and Algorithm 3. While matrix sizes change in FrodoKEM-976 and FrodoKEM-1344, the operational sequence remains unchanged.

The FrodoKEM scheme deviates from the standard LWE problem in its use of matrices instead of vectors for secrets and errors, introducing structural differences in key generation and encryption. In the FrodoKEM-640 reference implementation, matrix elements are stored as 16-bit unsigned integers (`uint16_t`), occupying 2 bytes each. This allows precise estimation of required bytes per matrix. Table I compares total

Algorithm 3 LWE - Decryption

Procedure DECRYPT(private_key, ciphertext)
  $(u, v) \leftarrow ciphertext$
  $s \leftarrow private\_key$
  $v - s \cdot u = m + e^T \cdot s' - s^T \cdot e' + e''$
  decrypted_message $\leftarrow m + e^T \cdot s' - s^T \cdot e' + e''$
  message $\leftarrow$ DECODE(decrypted_message)
  return message
End Procedure

TABLE I: Byte Estimation and Impact of Error Matrices Generation in FrodoKEM-640

FrodoKEM	Gen. Bytes (with errors)	Gen. Bytes (without errors)	Error Bytes Contribution
Key Generation	839,680	829,440	10,240 (1.22%)
Encryption	839,808	829,440	10,368 (1.23%)
Decryption	0	0	0

bytes generated *with* and *without* error matrices $\mathbf{e}, \mathbf{e}', \mathbf{e}''$.

In key generation, $\mathbf{e}$ accounts for $10{,}240$ bytes, about 1.22% of total data. Since both $\mathbf{s}$ and $\mathbf{e}$ are $10{,}240$ bytes, omitting $\mathbf{e}$ halves the Gaussian sampler load, reducing data from $20{,}480$ bytes to $10{,}240$ bytes.

In encryption, $\mathbf{s}'$ and $\mathbf{e}'$ ($10{,}240$ bytes each) are generated along with $\mathbf{e}''$ (128 bytes). Avoiding $\mathbf{e}'$ and $\mathbf{e}''$ reduces Gaussian sampling from $20{,}608$ bytes to $10{,}240$ bytes, a 50.3% reduction. However, considering total byte generation (including $\mathbf{A}$), the savings from omitting error matrices is about 1.22% per phase.

C. Approximate Computing

Approximate Computing (AxC) is a paradigm that relaxes the requirement for precise computations to improve efficiency in power consumption, execution speed, and resource usage. It is particularly useful in error-tolerant applications where sub-optimal results are sufficient. In this work, we explore approximate circuits from a different perspective: deliberately adding errors in a controlled manner. We use *approximate adders* (AxADDs in the following) available in the EvoApproxLib8b generated by means of genetic algorithms and whose approximation is performed at the functional level [10].

Classical metrics for approximate circuits, as described in [9], focus on the magnitude (absolute and percentage) of the error and its probability. However, these metrics are not sufficient for our needs, as they do not provide information on the error distribution for random inputs. This is crucial in FrodoKEM, where the required error must follow a Gaussian distribution, as introduced in subsection II-B. Therefore, we characterized the error distribution of AxADDs whose worst-case absolute error does not exceed the maximum specified in FrodoKEM. More details are provided in section III.

D. Related Works

In the literature, some works are available illustrating the concept of naturally adding the necessary errors while performing operations by means of *inexact computing* within the LWE context. Specifically, the authors of [11] propose a novel method of exploiting hardware glitches to perform noisy inner products, using these inaccuracies to implement the *Learning Parity with Noise* (LPN) problem, i.e., a simpler version of LWE based on binary data instead of integer arithmetic. Results are obtained via simulation of inner products architectures over which errors are induced by means of frequency and voltage over-scaling. In this way, the authors moved to what is defined as *Learning Parity with Physical Noise* (LPPN). An FPGA implementation has then been proposed in [12] analyzing advantages, drawbacks, and open questions of such solutions. As a further study, the authors of [13] focused on the LWE problem in order to potentially realize usable instances of *Learning With Physical Noise* (LWPN).

As far as we are aware at the time of this paper, no other work has been carried out in terms of applying these concepts to the most recent LWE-based schemes. Our proposal is fully based on digital circuits that are easier to control and not subject to environmental variables, such as temperature, as is the case for the aforementioned solutions. Furthermore, it can be applied to schemes that require deterministic error during decryption (such as FrodoKEM) since the error introduced by *approximate hardware* can be fully characterized.

III. Implementing FrodoKEM with AxAdders

This section details the sequence of techniques used to successfully implement the FrodoKEM scheme using approximate operators. Our objective is to modify the traditional structure of the scheme by eliminating the explicit generation and addition of external errors, while still preserving the necessary noise characteristics. A high-level overview is illustrated in Figure 1, where TRNG denotes a True Random Number Generator, HASH represents a secure hash function, and GS stands for Gaussian Sampler. To achieve this, we propose an approximate implementation of the $\mathbf{A}\cdot\mathbf{s}$ inner product, where the required error is inherently introduced during computation. The standard inner product that is computed during the $\mathbf{A}\cdot\mathbf{s}+\mathbf{e}$ operation can be formulated as follows:

$$b_{ij} = \sum_{k=0}^{n-1} a_{ik}s_{kj} + \underbrace{e_{ij}}_{\text{error term}} = a_{i0}s_{0j} + \cdots + a_{in}s_{nj} + e_{ij} \quad (3)$$

where the summation represents the inner product of the ith row of $\mathbf{A}$ and the jth column of $\mathbf{S}$, plus the addition of the error term e_{ij}.

Instead of explicitly adding a separately generated error term, as done in Equation 3, we embed the error directly into the arithmetic operations themselves. A preliminary analysis of the inner product structure revealed that the most effective and controllable way to introduce errors is through the use of an approximate adder (AxADD) in selected additions of the partial products. The AxADD is used when performing the last addition in the accumulation process of each b_{ij}, resulting in:

$$b_{ij} = \sum_{k=0}^{n-1} a_{ik}s_{kj} = a_{i0}s_{0j} + \ldots \underbrace{+}_{\text{AxADD}} a_{in}s_{nj} \quad (4)$$

This choice provides a controlled way to embed errors while maintaining computational efficiency. In LWE-based cryptographic schemes, such as FrodoKEM, the presence of error is fundamental for security, as discussed in section I. However, if the introduced errors have a large standard deviation σ, their accumulation during decryption (Algorithm 3) can mask the original message, rendering it indistinguishable from random noise and causing decryption failures. Therefore, it is crucial

to ensure that the added noise remains within well-defined bounds of σ, preserving the correctness of the scheme while still guaranteeing security. This requirement guided our selection of a suitable AxADD.

A. Choosing a suitable AxADD

A preliminary analysis has been carried out on the noise involved directly in the FrodoKEM reference implementation [14]. Running the NIST Known Answer Tests (KATs), the involved errors on each value of the public key $\mathbf{b}$ have been characterized and the result is shown in Figure 2a. For this and the following characterizations, a set of 5000 samples has been considered since a higher number had no relevant impact on the precision of the shown Gaussian fit.

An AxADD is designated to introduce controlled errors into computations. It is crucial to select an AxADD whose error distribution closely matches the required distribution for the cryptographic scheme. If the errors introduced by the AxADD deviate significantly from the expected noise distribution, either by having too small or too large σ, this can have detrimental effects on both security and correctness of the scheme. A first selection of adders from the EvoApproxLib8b [9] was made by focusing on the Worst-Case Absolute Error (WCAE), ensuring that it was as close as possible to 12, as specified by FrodoKEM-640. To retrieve the necessary error information, each selected AxADD was emulated by means of the provided C model.

The error extraction process consisted of generating two 16-bit random numbers. Two sums were then computed: one exact sum using the addition operator, and one approximate sum using the emulated AxADD function. The difference between the approximate and the exact sums provided the error figures that can be observed in the plots. Each plot displays the error values on the x-axis, and its frequency of occurrence on the y-axis. The goal was to select an AxADD whose error distribution had the minimal variation from the reference one in Figure 2a. Figure 2b shows the error distribution of the `addu16_0GN` adder from the EvoApproxLib8b library [9]. This adder was selected as the best trade-off in our analysis, as its error distribution closely resembles the reference one. Figure 2c provides a direct comparison between the error distributions of `addu16_0GN` and FrodoKEM-640. While `addu16_0GN` exhibits a slightly lower standard deviation σ, its performance remains within an acceptable range. A theoretical discussion of its implications is provided in subsection III-B.

B. Decryption Failure Rate

The reference metric for the maximum acceptable error is the *Decryption Failure Rate* (DFR), that is defined as the probability that a *decryption* operation will fail to correctly recover the original *plaintext* from the *ciphertext*. In FrodoKEM, since the error follows a discrete Gaussian distribution, the DFR can be estimated by means of *Gaussian Tail Bound* (GTB) for *subgaussians*[1] [15], that is the probability that a discrete Gaussian random variable (the error) exceeds a given threshold. Adapting the GTB formula to FrodoKEM it follows:

$$DFR \approx 2^{1+\frac{-\pi\cdot\frac{(\frac{q}{4})^2}{\sigma_{tot}^2}}{\ln(2)}} \approx 2^{-3868} \tag{5}$$

where for FrodoKEM-640 $q = 2^{15}$ and $\sigma_{tot} = 2\cdot 640\cdot\sigma^4+\sigma^2$, with $\sigma = 2.8$. The total variance formula was derived considering all the components of the error present in the decrypted message in Algorithm 3.

When using the `addu16_0GN` adder the total error variance formula slightly changes to $\sigma_{tot} = 2\cdot 640\cdot\sigma_{ax}^2\sigma^2+\sigma_{ax}^2$, since it now depends on both the discrete Gaussian distribution of $\mathbf{s}$ and the one of $\mathbf{e}$ directly coming from the AxADD. For this setup, $DFR \approx 2^{-3811}$, which is slightly higher than the reference FrodoKEM-640 implementation in Equation 5, but still far lower than the $2^{-138.7}$ limit of the specification.

C. AxFrodoKEM-640 Implementation

Our approximate FrodoKEM-640 software implementation is directly derived from the reference one [14]. We acted on the sections of code involving the error generation and addition in the *key generation* and *encryption* functions. A graphical high-level overview of transitioning to an approximate version of the algorithm is available in Figure 1 for clarity. The diagram represents the standard algorithm implementation, with the greyed-out components highlighting the error generation chain that is removed due to the use of AxADD.

More specifically, all the function calls to the SHAKE128 primitive to generate the matrix $\mathbf{e}$ were removed, having a direct impact on performances. To introduce the necessary error in the scheme, we acted on the inner product $\mathbf{A}\cdot\mathbf{s}$. As anticipated, Equation 4 shows the computation details for each element b_{ij} of the public matrix $\mathbf{b}$. Each partial product is normally cumulated, with the exception of the last one, which is added to the partial result using the `addu16_0GN` adder introduced in subsection III-A. Its operation has been emulated by means of a C function call provided by the authors of [9]. In this way, the errors do not need to go through the Gaussian sampling process, resulting in further performance improvements. The rest of the algorithm follows the reference implementation, except for less generated pseudo-random data and using reduced data structures, given the smaller overall number of bytes to be stored. This approximate version of FrodoKEM-640 has been finally tested against the NIST KATs to check its proper functionality and performance.

IV. Results

Table II reports the gains obtained by approximate FrodoKEM-640. This study was performed on the reference FrodoKEM-640 C implementation [14] enriched with the modifications described in subsection III-C. The study was conducted on a Linux system with kernel version 4.18.0, an

[1] A subgaussian distribution is a probability distribution whose tail probabilities decay at least as fast as a Gaussian distribution, e.g. a discrete Gaussian distribution.

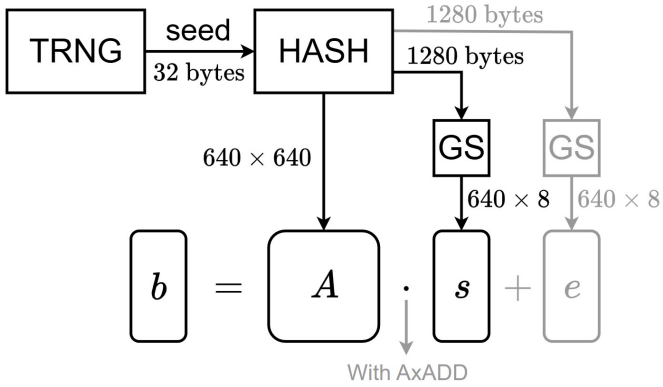


Fig. 1: Removing the error generation chain.

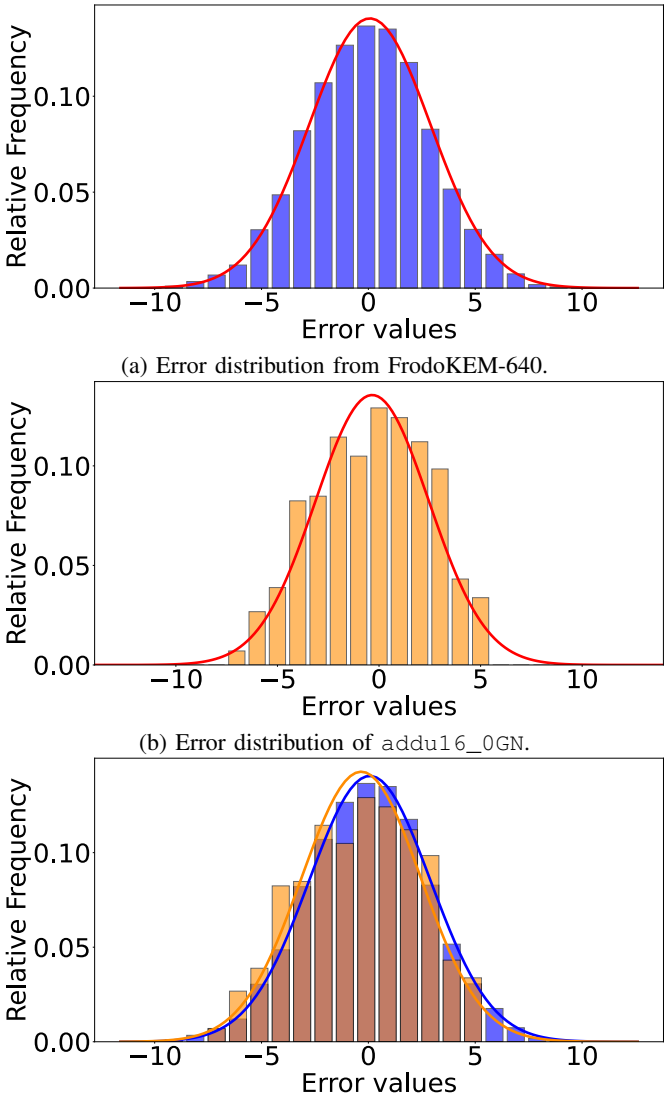
Intel Core i3-2120 CPU @ 3.30GHz, 16GB of RAM, and GCC version 8.5.0. The reported metrics refer to averaged measures obtained from multiple runs of the test vectors. Each clock cycle measure has been performed by reading the Time Stamp Counter register of the CPU.

The first three columns of Table II show the execution time (in clock cycles) for each of the main algorithms (namely, key generation, encryption, and decryption). The last column indicates the execution time for a complete run of FrodoKEM. The data for the Average Cycles regarding the approximate implementation (namely, labeled with AxADD) includes one additional processing step. Since the AxADD was emulated in software, the performance improvement could not be directly appreciated because an AxADD instruction in this setup would take 20 to 30 times more clock cycles than a simple ADD. Therefore, the total number of clock cycles was computed assuming an AxADD is implemented on board, substituting their execution time with that of a single operation. Values in parentheses highlight the percentage gain of the approximate FrodoKEM-640 implementation over the reference one.

It is evident that the reduction in clock cycles is greater than the reduction in generated bytes. This is due to the fact that error generation carries some overhead, as it does not consist of a simple random data sampling. Extracting pseudo-random bytes from the SHAKE128 function also implies additional function calls for its correct setup and data extraction. Moreover, after these bytes are generated, they must go through Gaussian sampling before being employed as errors. Consequently, the resulting byte savings of up to 1.23% translate into a reduction in execution time of up to 2.19%. When considering an entire run of FrodoKEM-640, these savings amount to 1.22% and 2.04% in bytes and execution time, respectively.

V. Discussion and Further Perspectives

With section III and section IV, we showed that it is possible to implement PQC algorithms via *AxC*. The main goal of this paper is to demonstrate that exploiting *approximate operators* to generate errors in LWE-based schemes is a valid and promising approach for more optimized implementations, while also achieving a slight performance improvement of 2.19% for FrodoKEM-640. In fact, the potential for gain in



(a) Error distribution from FrodoKEM-640.

(b) Error distribution of `addu16_0GN`.

(c) Comparison of both distributions.

Fig. 2: Relevant error distributions with Gaussian fits. (a) Error distribution extracted from the reference FrodoKEM-640 C implementation. (b) Error distribution of the `addu16_0GN` adder. (c) Direct comparison of both distributions, with FrodoKEM-640 in purple and `addu16_0GN` in orange.

the FrodoKEM algorithm is limited because the public matrix **A** is much larger than other components. This algorithm was chosen due to its close relationship with the original LWE problem, allowing us to potentially extend such concepts to the broadest set of algorithms possible. Moreover, as a byproduct of our technique, we effectively remove the need to explicitly sample half the Gaussian errors originally required, reducing the total number of Gaussian samplings by 50.3%.

To further illustrate the potential of this idea, we briefly consider another LWE-based scheme, ML-KEM-512 [16], whose structure slightly differs from FrodoKEM and allows for greater optimization in error generation. In this case, saved bytes reach 14.14%, as shown in Table III. This highlights

TABLE II: AxFrodoKEM-640. Execution time in kilo clock cycles (kcc) and required number of bytes for error generation (B) for both reference implementation and with AxADD.

	KeyGen	Encrypt	Decrypt	FrodoKEM
Execution Time Reference (kcc)	55,756	72,418	73,215	201,388
Execution Time AxADD (kcc)	54,580 (-2.11%)	71,078 (-1.85%)	71,078 (-2.19%)	197,272 (-2.04%)
Error Generation Reference (B)	839,680	839,808	850,048	2,529,536
Error Generation AxADD (B)	829,440 (-1.22%)	829,440 (-1.23%)	839,680 (-1.22%)	2,498,560 (-1.22%)

TABLE III: Estimated Bytes and Impact of Error Matrix Generation in ML-KEM-512

ML-KEM	Gen. Bytes (with errors)	Gen. Bytes (without errors)	Error Bytes Contribution
Key Generation	2664	2280	384 (14.14%)
Encryption	2664	2280	384 (14.14%)
Decryption	0	0	0

the potential of AxC in significantly enhancing the efficiency of PQC schemes with different designs. As a supplementary note, we acknowledge that the absence of random error sampling prior to the Gaussian sampling introduces a reduction in entropy within the scheme. While a detailed quantitative analysis of this effect is beyond the scope of this study, it is not included in the present work. Nonetheless, this aspect does not undermine the validity of our approach and could be addressed by randomizing the location of the AxADD in Equation 4. We identify this as a valuable direction for future research. Additionally, future work will investigate practical hardware implementations of AxC techniques in PQC applications.

In summary, this paper establishes a proof of concept for the integration of AxC into PQC algorithms, demonstrating its viability and opening the door to future enhancements in performance and efficiency. The approach presented here lays the groundwork for further exploration into refined optimization strategies and practical implementation techniques, with promising potential for advancing the state of PQC through approximate computing.

VI. Conclusion

In conclusion, this paper demonstrates the feasibility of using Approximate Computing (AxC) to optimize Post-Quantum Cryptography (PQC) algorithms, using the FrodoKEM scheme as a case study. By employing approximate adders (AxADDs) to introduce the necessary errors, we presented a working AxC-based implementation that serves as a valid proof of concept for this approach. As an additional outcome, we achieved a performance improvement of 2.19% in execution time and a 1.23% reduction in generated bytes. Notably, this corresponds to an overall 50.3% reduction in the Gaussian sampled data.

Although the gains in FrodoKEM-640 are limited by the large size of the public matrix $\mathbf{A}$, this work paves the way for more significant improvements in other PQC schemes, such as ML-KEM-512, where the potential savings in bytes can reach up to 14.14%. Future research will focus on developing practical hardware implementations of AxC techniques in PQC, as well as investigating methods to address the reduced entropy, such as enhancing the randomness of the AxADD operations. This study underscores the potential of AxC in enhancing the efficiency of PQC algorithms and opens avenues for further research in this promising area.

Acknowledgment

This work was supported by the French National Research Agency (ANR) via the CARNOT LIST AxPQC funding.

References

[1] P. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.

[2] E. Alkim, J. W. Bos, L. Ducas, P. Longa, I. Mironov, M. Naehrig, V. Nikolaenko, C. Peikert, A. Raghunathan, and D. Stebila, "FrodoKEM: Learning with errors key encapsulation." Submission to the NIST PQC Standardization Project, 2021. Available at: https://frodokem.org/files/FrodoKEMspecification-20210604.pdf.

[3] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," in *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '05, (New York, NY, USA), p. 84–93, Association for Computing Machinery, 2005.

[4] National Institute of Standards and Technology (U.S.), "SHA-3 standard : Permutation-based hash and extendable-output functions," 2015.

[5] G. Bertoni, J. Daemen, M. Peeters, and G. V. Assche, "Keccak," in *International Conference on the Theory and Application of Cryptographic Techniques*, 2013.

[6] J. Howe, A. Khalid, C. Rafferty, F. Regazzoni, and M. O'Neill, "On practical discrete gaussian samplers for lattice-based cryptography," *IEEE Transactions on Computers*, vol. 67, no. 3, pp. 322–334, 2018.

[7] D. S. Crescenzo, R. C. Rodriguez, R. Alidori, F. Bruguier, E. Valea, P. Benoit, and A. Bosio, "Hardware accelerator for fips 202 hash functions in post-quantum ready socs," *2024 IEEE 30th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, pp. 1–6, 2024.

[8] J. Han and M. Orshansky, "Approximate computing: An emerging paradigm for energy-efficient design," in *2013 18th IEEE European Test Symposium (ETS)*, pp. 1–6, 2013.

[9] V. Mrazek, R. Hrbacek, Z. Vasicek, and L. Sekanina, "Evoapprox8b: Library of approximate adders and multipliers for circuit design and benchmarking of approximation methods," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, pp. 258–261, 2017.

[10] V. Mrazek, "Optimization of bdd-based approximation error metrics calculations," in *2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pp. 86–91, 2022.

[11] D. Kamel, F.-X. Standaert, A. Duc, D. Flandre, and F. Berti, "Learning with physical noise or errors," *IEEE Transactions on Dependable and Secure Computing*, vol. 17, no. 5, pp. 957–971, 2020.

[12] D. Bellizia, C. Hoffmann, D. Kamel, H. Liu, P. Méaux, F.-X. Standaert, and Y. Yu, "Learning parity with physical noise: Imperfections, reductions and fpga prototype," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, p. 390–417, Jul. 2021.

[13] D. Bellizia, C. Hoffmann, D. Kamel, P. Méaux, and F.-X. Standaert, "When bad news become good news: Towards usable instances of learning with physical errors," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, Issue 4, pp. 1–24, 2022.

[14] E. Alkim, J. W. Bos, L. Ducas, P. Longa, I. Mironov, M. Naehrig, V. Nikolaenko, C. Peikert, A. Raghunathan, and D. Stebila, "Pqcrypto-lweke." https://github.com/microsoft/PQCrypto-LWEKE, 2021. Accessed: Mar. 5, 2025.

[15] C. Peikert, *A Decade of Lattice Cryptography*. Now Foundations and Trends, 2016.

[16] National Institute of Standards and Technology (U.S.), "Module-lattice-based key-encapsulation mechanism standard," 2024. Available at: https://doi.org/10.6028/NIST.FIPS.203.